

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Reporte #8 - Semana 9 – Sivarama - Capítulo 13
Modos de direccionamiento

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 23/09/2024

Bibliografía:

[1] S. P. Dandamudi, *Guide to Assembly Language Programming in Linux*. Springer Science & Business Media, 2005, pp. 273–289.

Abstract—*In assembly language, addressing modes determine how the location of an operand is specified in an instruction. This paper explores key addressing modes, including register addressing, where operands are stored in processor registers; immediate addressing, where operands are embedded within the instruction itself; and various memory addressing modes such as base, indexed, and base-indexed addressing. These memory modes allow efficient access to complex data structures like arrays and records. Additionally, differences between 16-bit and 32-bit addressing modes are discussed, highlighting the flexibility and expanded functionality offered by 32-bit systems.*

I. Modos de direccionamiento

En lenguaje ensamblador, el modo de direccionamiento especifica cómo se indica la ubicación de un operando necesario para una instrucción. Existen diferentes maneras de hacerlo, ya sea en un registro, en la instrucción misma, en la memoria o en un puerto de E/S. Este capítulo se enfoca en los modos de direccionamiento para operandos ubicados en registros, memoria o la propia instrucción.

- **Modo de direccionamiento por registro:** Los operandos son proporcionados y almacenados en los registros del procesador. Es el más eficiente, ya que minimiza la demora en acceder a los operandos.
- **Modo de direccionamiento inmediato:** El operando se codifica directamente en la instrucción, estando disponible tan pronto como se lea.
- **Modos de direccionamiento de memoria:** Los operandos en la memoria requieren una dirección lógica, que se compone de una base de segmento y un desplazamiento (llamado dirección efectiva).

II. Modos de direccionamiento en memoria

Estos modos permiten acceder a operandos en memoria de diversas formas. Dependen del tamaño de la dirección (16 o 32 bits) y soportan estructuras y arreglos en lenguajes de alto nivel.

- **Dirección base:** Suma el contenido de un registro base con un desplazamiento firmado para calcular la dirección efectiva. Es útil para acceder a elementos dentro de estructuras, como en un registro de un curso, donde la base señala el inicio y el desplazamiento apunta a un campo específico.

Ejemplo:

```
mov EAX, [EBX+2] ; Lee el campo a partir de un desplazamiento
```

- **Dirección indexada:** Suma el contenido de un registro índice multiplicado por un factor de escala (1, 2, 4 u 8) con un desplazamiento para obtener la dirección efectiva. Se usa comúnmente para acceder a elementos en arreglos.

Ejemplo:

```
mov EAX, [marks_table+ESI*4] ; Accede a un elemento en un arreglo de 4 bytes
```

- **Dirección base-indexada:** Suma un registro base, un índice y un desplazamiento para obtener la dirección efectiva. Es útil para acceder a arreglos bidimensionales o arreglos de registros.

Ejemplo:

```
mov EAX, [EBX+ESI] ; Accede a dos elementos consecutivos en un arreglo
```

III. Arrays

Los arreglos son estructuras fundamentales para organizar datos relacionados. Pueden ser unidimensionales o multidimensionales.

- **Arreglos unidimensionales:** En lenguajes de alto nivel, como C, los arreglos se declaran con un nombre, número de elementos, tamaño del elemento, tipo de elemento y rango de índices. En ensamblador, los arreglos se implementan asignando el espacio de almacenamiento necesario.

Ejemplo en C:

```
int test_marks[10];
```

Esto crea un arreglo de 10 enteros. En ensamblador, se puede declarar así:

```
test_marks resd 10 ; Reserva espacio para 10 enteros
```

Los elementos se almacenan linealmente en la memoria.

- **Arreglos multidimensionales:** Son más complejos de manejar en ensamblador, ya que requieren cálculos adicionales para acceder a elementos. En un arreglo bidimensional, el acceso a un elemento se realiza utilizando tanto un registro base como un registro índice, lo que facilita la navegación entre filas y columnas.

IV. Diferencias entre direccionamiento de 16 y 32 bits

Las direcciones de 16 y 32 bits difieren en la flexibilidad y en los registros que pueden utilizarse. En 32 bits, los registros de propósito general pueden ser EAX, EBX, ECX, EDX, ESI, EDI, EBP, y ESP. Además, se pueden usar factores de escala de 1, 2, 4 u 8 para direccionar más eficientemente los arreglos de datos grandes.

El tamaño de las direcciones se determina mediante el bit D en el descriptor del segmento CS, aunque se pueden modificar utilizando los prefijos de sobrecarga de tamaño:

- 66H para cambiar el tamaño del operando.
- 67H para cambiar el tamaño de la dirección.

Por ejemplo, la instrucción `mov EAX, [BX]` inserta automáticamente el prefijo de sobrecarga de tamaño de dirección 67H cuando se usa en un sistema de 32 bits.